

گزارش پروژه ۳ - نرم افزار ریاضی

سعد ملائی شماره دانشجویی: ۴۰۱۱۳۰۲۷

تیر ۱۴۰۵

۱ درونیابی لاگرانژ و اسپلاین مکعبی

در این بخش، هدف درونیابی نقاط داده شده زیر با استفاده از دو روش چندجمله‌ای لاگرانژ و اسپلاین مکعبی است:

$$\{(1, 1), (2, 3), (3, 5), (4, 8), (5, 5), (6, 2)\}$$

۱.۱ کد پایتون پیاده‌سازی

برای محاسبات نمادین و استخراج ضابطه دقیق از کتابخانه SymPy و برای محاسبه ضرایب اسپلاین از کتابخانه SciPy استفاده شده است:

```
1 import numpy as np
2 import sympy as sp
3 from scipy.interpolate import CubicSpline
4
5 x_vals = np.array([1, 2, 3, 4, 5, 6])
6 y_vals = np.array([1, 3, 5, 8, 5, 2])
7 x = sp.Symbol('x')
8
9 # 1. Lagrange Interpolation
10 lagrange_poly = 0
11 for i in range(len(x_vals)):
12     term = y_vals[i]
13     for j in range(len(x_vals)):
14         if i != j:
15             term *= (x - x_vals[j]) / (x_vals[i] - x_vals[j])
16     lagrange_poly += term
17 exact_lagrange = sp.expand(lagrange_poly)
18 print("Lagrange:", exact_lagrange)
19
20 # 2. Cubic Spline
21 cs = CubicSpline(x_vals, y_vals)
22 for i in range(len(x_vals) - 1):
23     a, b, c, d = cs.c[:, i]
24     xi = x_vals[i]
25     spline_eq = a*(x - xi)**3 + b*(x - xi)**2 + c*(x - xi) + d
26     print(f"Interval [{xi}, {x_vals[i+1]}]:", sp.expand(spline_eq))
```

۲.۱ ضابطه دقیق ریاضی حاصل از خروجی

۱.۲.۱ الف) چندجمله‌ای لاگرانژ

با بسط و ساده‌سازی عبارات نمادین در پایتون، چندجمله‌ای لاگرانژ درجه ۵ حاصل به صورت زیر است:

$$P(x) = -0.1167x^5 + 2.0833x^4 - 13.9167x^3 + 43.4167x^2 - 61.4667x + 31.0$$

۲.۲.۱ ب) اسپلاین مکعبی

ضابطه چندبرخی اسپلاین مکعبی بر اساس بازه‌های مختلف به صورت معادلات درجه ۳ زیر استخراج شد:

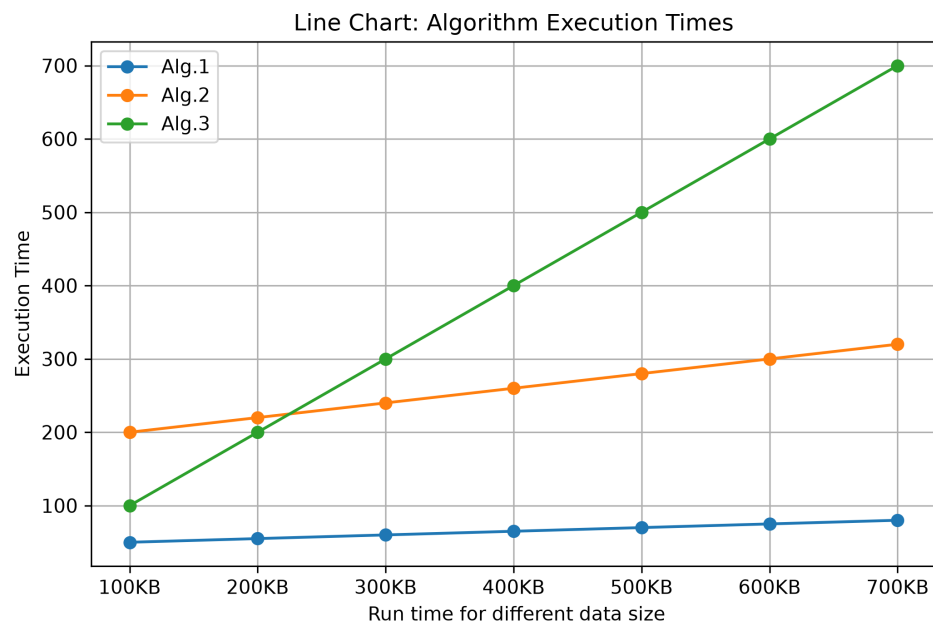
$$S(x) = \begin{cases} -0.45x^3 + 1.35x^2 + 1.1x - 1.0 & x \in [1, 2] \\ 0.55x^3 - 4.65x^2 + 13.1x - 9.0 & x \in [2, 3] \\ -1.35x^3 + 12.45x^2 - 38.2x + 42.3 & x \in [3, 4] \\ 1.05x^3 - 16.35x^2 + 77.0x - 111.3 & x \in [4, 5] \\ 0.2x^3 - 3.6x^2 + 13.25x - 5.0 & x \in [5, 6] \end{cases}$$

۲ تحلیل فایل اکسل و رسم نمودارها

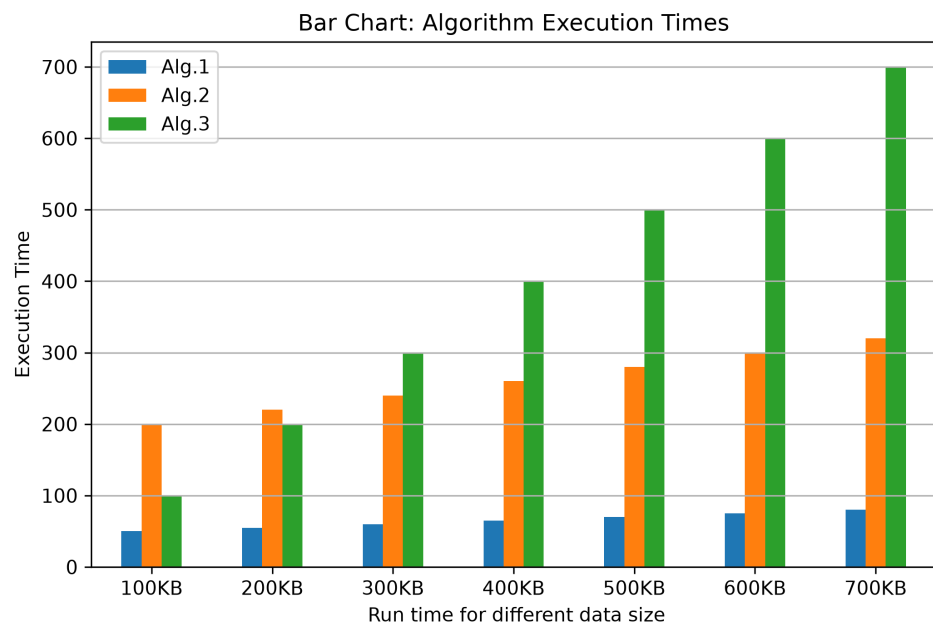
در این بخش، زمان اجرای ۳ الگوریتم مختلف به ازای داده‌های ورودی ۱۰۰ الی ۷۰۰ کیلوبایت (با احتساب داده جدید بند ب) به صورت پویا پردازش شده است.

۱.۲ خلاصه کد پایتون پیاده‌سازی

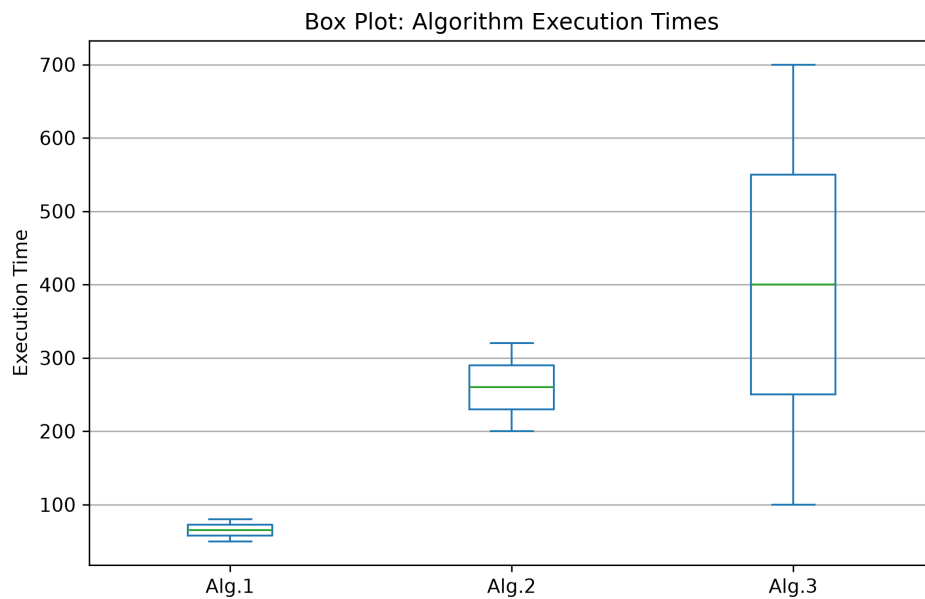
```
1 # Read all the data
2 df = pd.read_excel(file_name)
3
4 # Extract
5 x_col = df.columns[0]
6 alg_cols = df.columns[1:]
7
8 # 1. Draw Line Plot
9 df.plot(x=x_col, y=alg_cols, kind='line', marker='o', figsize=(8, 5))
10 plt.title('Line Chart: Algorithm Execution Times')
11 plt.ylabel('Execution Time')
12 plt.xlabel(x_col)
13 plt.grid(True)
14 plt.show()
15
16 # 2. Draw Bar Plot
17 df.plot(x=x_col, y=alg_cols, kind='bar', figsize=(8, 5))
18 plt.title('Bar Chart: Algorithm Execution Times')
19 plt.ylabel('Execution Time')
20 plt.xlabel(x_col)
21 # Keep X-axis labels horizontal
22 plt.xticks(rotation=0)
23 plt.grid(axis='y')
24 plt.show()
25
26 # 3. Draw Box Plot
27 # Provide only numerical columns (algorithms) for the box plot
28 df[alg_cols].plot(kind='box', figsize=(8, 5))
29 plt.title('Box Plot: Algorithm Execution Times')
30 plt.ylabel('Execution Time')
31 plt.grid(axis='y')
32 plt.show()
33
34 # the average for algorithm 2 for 100-600 KB.
35 df_filtered = df[~df[x_col].astype(str).str.contains('700')]
36 if 'Alg.2' in df_filtered.columns:
37     avg_alg2 = df_filtered['Alg.2'].mean()
38     print("--- Output for Part C ---")
39     print(f"Average execution time of Algorithm 2 (for 100-600 KB data):  
        {avg_alg2}")
40 else:
41     print("Column named 'Alg.2' not found in the Excel file. Please check  
        the column names.")
```



شکل ۱: نمودار خطی تغییرات زمان اجرای الگوریتم‌ها بر حسب حجم داده



شکل ۲: نمودار میله‌ای مقایسه‌ای زمان اجرای سه الگوریتم



شکل ۳: نمودار جعبه‌ای توزیع داده‌های زمان اجرای هر الگوریتم

۳.۲ تحلیل نمودارها

با مشاهده نمودارها می‌توان نتیجه گرفت که با افزایش حجم داده‌ها، زمان اجرای هر سه الگوریتم روند صعودی دارد، اما نرخ رشد آن‌ها کاملاً متفاوت است. الگوریتم ۱ بالاترین کارایی را داشته و کمترین حساسیت را نسبت به افزایش حجم داده نشان می‌دهد (به عنوان مثال، در حجم ۷۰۰ کیلوبایت زمان اجرای آن تنها ۸۰ است). الگوریتم ۲ عملکردی میانی و رشدی متوسط دارد (زمان اجرای ۳۲۰ برای حجم ۷۰۰ کیلوبایت). در مقابل، الگوریتم ۳ با شیب تندی افزایش یافته و بیشترین زمان اجرا را به خود اختصاص می‌دهد (زمان اجرای ۷۰۰ برای ۷۰۰ کیلوبایت). در نتیجه، برای پردازش داده‌های حجیم‌تر، الگوریتم ۱ بهینه‌ترین و الگوریتم ۳ نامناسب‌ترین انتخاب محسوب می‌شود.

۴.۲ میانگین زمان اجرای الگوریتم ۲

با فیلتر کردن سطر مربوط به داده‌های ۷۰۰ کیلوبایت، میانگین زمان اجرای الگوریتم ۲ صرفاً برای حجم داده‌های ۱۰۰ تا ۶۰۰ کیلوبایت از روی فایل اکسل محاسبه شد که مقدار آن برابر است با:

$$\text{Average} = 250.0$$

۳ انتگرال گیری عددی تابع $f(x) = e^{x^2}$

در این قسمت، مقدار انتگرال معین تابع فوق در بازه $(0, 1)$ با روش‌های عددی مختلف محاسبه شده است.

۱.۳ کد پایتون پیاده‌سازی

```
1 import numpy as np
2 from scipy.integrate import trapezoid, simpson, fixed_quad, quad
3
4 def f(x): return np.exp(x**2)
5 a, b = 0, 1
6 x_vals = np.linspace(a, b, 101)
7 y_vals = f(x_vals)
8
9 print("Trapezoidal:", trapezoid(y_vals, x_vals))
10 print("Simpson's:", simpson(y_vals, x=x_vals))
11 integral_gauss, _ = fixed_quad(f, a, b, n=5)
12 print("Gaussian:", integral_gauss)
13 integral_exact, abs_error = quad(f, a, b)
14 print("Reference:", integral_exact)
```

۲.۳ نتایج و تحلیل مقایسه‌ای

نتایج محاسبات عددی به دست آمده با دقت بالا به شرح زیر است:

□ روش دوزنقه‌ای (با ۱۰۰ بازه): 1.46371405

□ روش سیمپسون (با ۱۰۰ بازه): 1.46268143

□ روش انتگرال گیری گوسی ($n = 5$): 1.46265172

□ مقدار دقیق انتگرال (مرجع): 1.46265175

تحلیل و ارزیابی تفصیلی:

تابع $f(x) = e^{x^2}$ به دلیل تقعر رو به بالا و رشد سریع در بازه $(0, 1)$ ، چالش مناسبی برای سنجش روش‌های انتگرال گیری است. روش دوزنقه‌ای به دلیل تقریب زدن تابع با خطوط راست، دارای خطای برش بالاتری است و مقدار انتگرال را بزرگتر از واقعیت محاسبه کرده است.

روش سیمپسون با تقریب سهموی (درجه ۲) روی بازه‌ها، انحنای تابع را بسیار بهتر پوشش داده و به جواب واقعی نزدیک شده است. در نهایت، روش رباعی گوسی تنها با استفاده از ۵ نقطه ارزیابی، به دلیل انتخاب هوشمندانه نقاط و وزن‌ها، به طور خیره‌کننده‌ای دقیق‌ترین پاسخ را ارائه داده و تا ۷ رقم اعشار با مقدار دقیق مرجع مطابقت دارد. این امر برتری این روش را در توابع هموار نشان می‌دهد.

۴ مخزن گیت‌هاب

تمامی کدهای توسعه داده شده در این پروژه به همراه پروژه‌های پیشین در مخزن گیت‌هاب قرار گرفته و کامیت‌های لازم انجام شده است. لینک عمومی مخزن در زیر قرار داده شده است:

لینک گیت‌هاب